EXPERIMENT 3

Study of C Programming Debugging Tools

Introduction

Debugging is an important step in software development. While writing a program, programmers may unintentionally introduce mistakes called **errors** or **bugs**. These bugs may cause incorrect results, program crashes, or unexpected behaviour. The process of identifying, analysing, and removing these bugs is known as **debugging**. In C programming, debugging can be performed using compiler warnings, output statements like printf(), and powerful debugging tools such as GNU Debugger (GDB).

Basic Terminology

Error

An error is a mistake made by the programmer while writing the source code. Errors may prevent the program from compiling or may produce incorrect results during execution.

Bug

A bug is a defect in a program that causes unexpected or incorrect behavior even though the program may compile successfully.

Debugging

Debugging is the systematic process of locating, analyzing, and fixing bugs in a computer program to ensure that it produces the expected output.

Types of Errors in C Programming

1. Syntax Errors

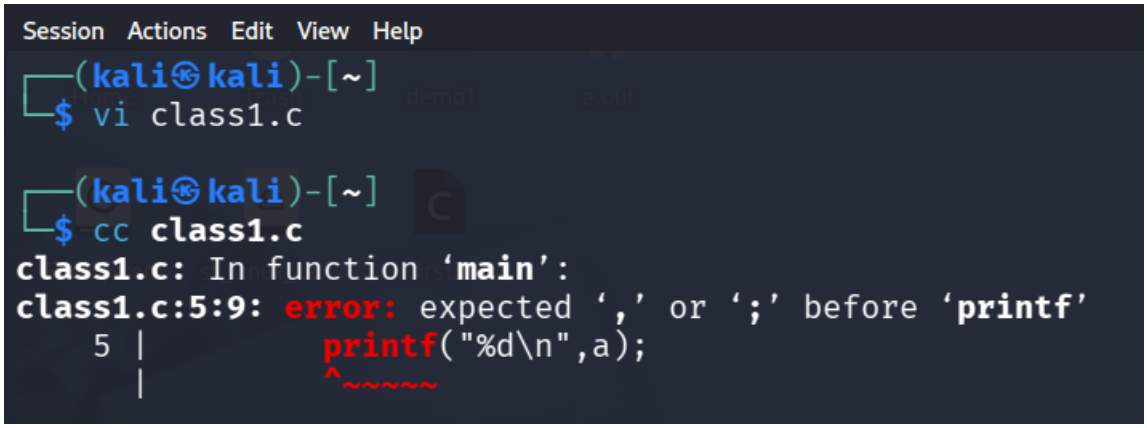
These errors occur when the program violates the grammatical rules of the C language. Examples include missing semicolons, unmatched braces, or incorrect keywords. The compiler detects these errors and stops compilation.

2. Compilation (Type) Errors

These occur when incompatible data types are used or when variables/functions are not declared correctly. Such errors are reported during compilation.

EXPERIMENT 3

Study of C Programming Debugging Tools

- 

3. Linker Errors

Linker errors occur after successful compilation when the linker cannot combine object files correctly, usually because a function is declared but not defined or a required library is missing.

```
#include <stdio.h>

void display();   // Function declaration

int main()
{
   display();    // Function call
   return 0;
}
```

The function display() is declared but **its definition is missing.** The compiler compiles the code, but the linker cannot find the actual function implementation and produces an error such as:

```
undefined reference to 'display'
 collect2: error: ld returned 1 exit status
```

4. Runtime Errors

These errors occur while the program is executing. Examples include division by zero, invalid memory access, or segmentation faults.

EXPERIMENT 3

Study of C Programming Debugging Tools

```
#include <stdio.h>

int main()
{
   int a = 10;
   int b = 0;

   int result = a / b;   // Division by zero

   printf("%d", result);

   return 0;
}
```

The program compiles successfully but crashes during execution because division by zero is not allowed.
Possible output:
Floating point exception (core dumped)

5. Logical Errors

Logical errors are the most difficult to detect because the program compiles and runs successfully but produces incorrect output due to incorrect program logic.

```
#include <stdio.h>

int main()
{
   int a = 10, b = 5;
   int sum;

   sum = a - b;    // Should be a + b

   printf("Sum = %d", sum);

   return 0;
}
```

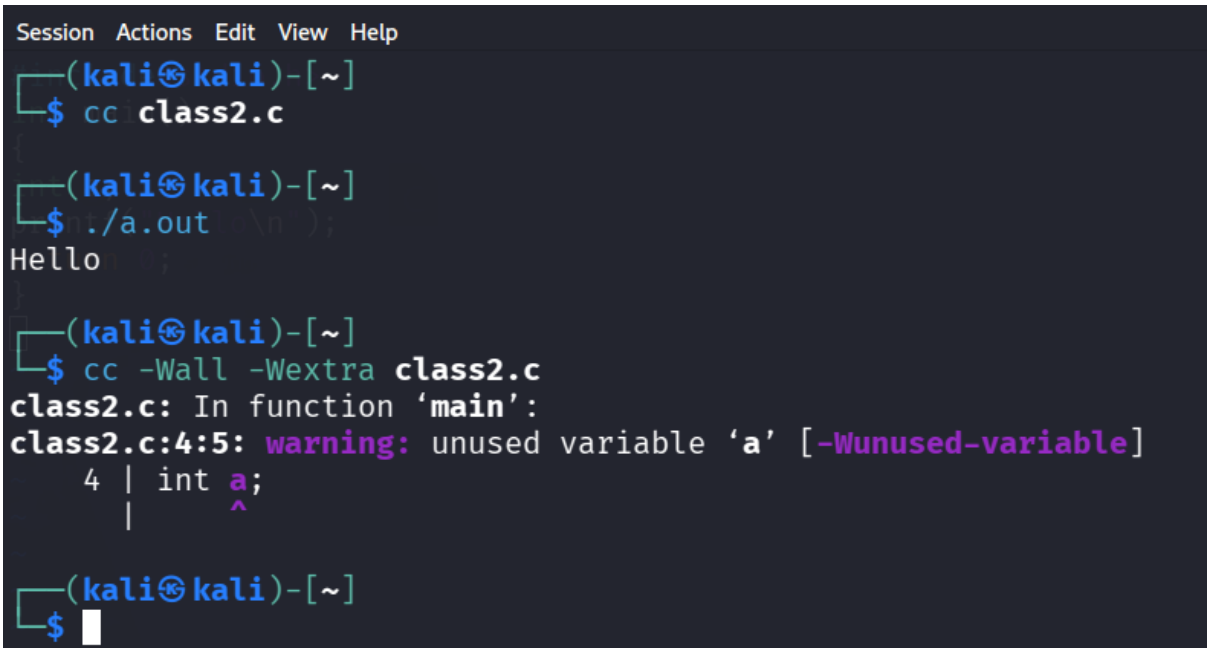
The program compiles and runs successfully, but it produces the wrong output because the programmer used the subtraction operator (-) instead of the addition operator (+).

Expected Output
Sum = 15
Actual Output
Sum = 5

CASE 2 :

Compiler Warning Flags

The GCC compiler provides warning options such as **-Wall** and **-Wextra**. These options help programmers identify suspicious code that may not produce compilation errors but could lead to bugs during program execution. Using these warning flags improves code quality and reduces programming mistakes.



- -Wall : Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall .

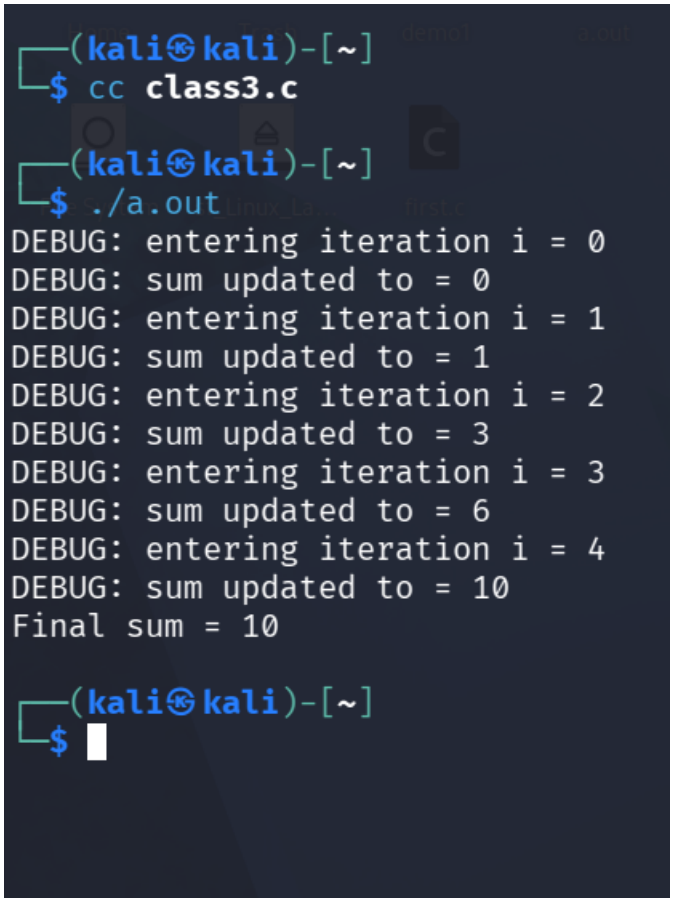
CASE 3 :

printf() Debugging

`printf()` debugging is one of the simplest debugging techniques. By printing the values of variables and the flow of execution, programmers can observe how the program behaves and locate logical errors without using advanced debugging tools.

EXPERIMENT 3

Study of C Programming Debugging Tools



GNU Debugger (GDB)

GNU Debugger (GDB) is a command-line debugging tool used to analyze C programs. It allows programmers to execute programs step by step, inspect variable values, set breakpoints, and identify runtime errors. Programs are usually compiled with the **-g** option before debugging so that debugging information is included in the executable.

Common GDB Commands

Command	Purpose
break	Sets a breakpoint at a specific line or function
run	Starts execution of the program
next	Executes the next line without entering functions
step	Executes the next line and enters function calls

EXPERIMENT 3

Study of C Programming Debugging Tools

Command	Purpose
Print	Displays the value of a variable
display	Automatically shows variable values after each step
continue	Continues execution until the next breakpoint
backtrace	Displays the sequence of function calls
quit	Exits the GDB debugger
List	prints sourcs code lines around the current execution position
Info locals	lists all the local variables and their current values in the current frame .
info breakpoints	Displays all active breakpoints and their numbers.
delete	Deletes the specified breakpoint number (or all if omitted).
disable	Temporarily disables a breakpoint without deleting it.
enable	Re-enables a previously disabled breakpoint.
quit	Exits the GDB debugging session

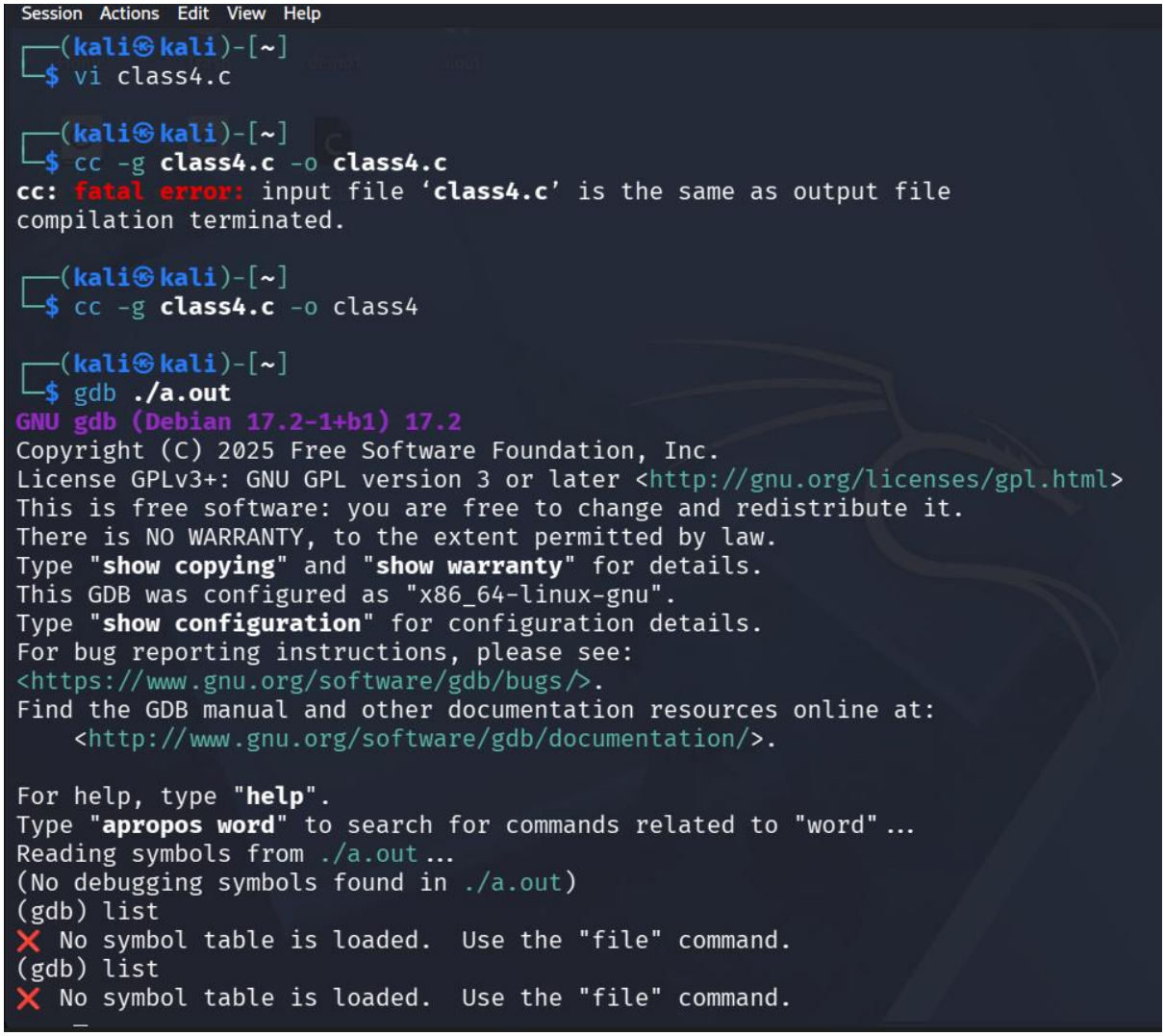
Case 4 – Debugging Logical Error using GDB

Expected output : 15(1+2+3+4+5)

Actual output : 10(1+2+3+4)

EXPERIMENT 3

Study of C Programming Debugging Tools



EXPERIMENT 3

Study of C Programming Debugging Tools

```
┌──(kali㉿kali)-[~]
└─$ gdb ./class4
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./class4...
(gdb) list
1       #include <stdio.h>
2       int main()
3       {
4               int i, sum=0;
5               for(i=1;i<5;i++)
6               {
7                       sum = sum + i;
8               }
9               printf("Sum = %d\n", sum);
10              return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file class4.c, line 7.
(gdb) run
Starting program: /home/kali/class4
[Thread debugging using libthread_db enabled]
```

EXPERIMENT 3

Study of C Programming Debugging Tools

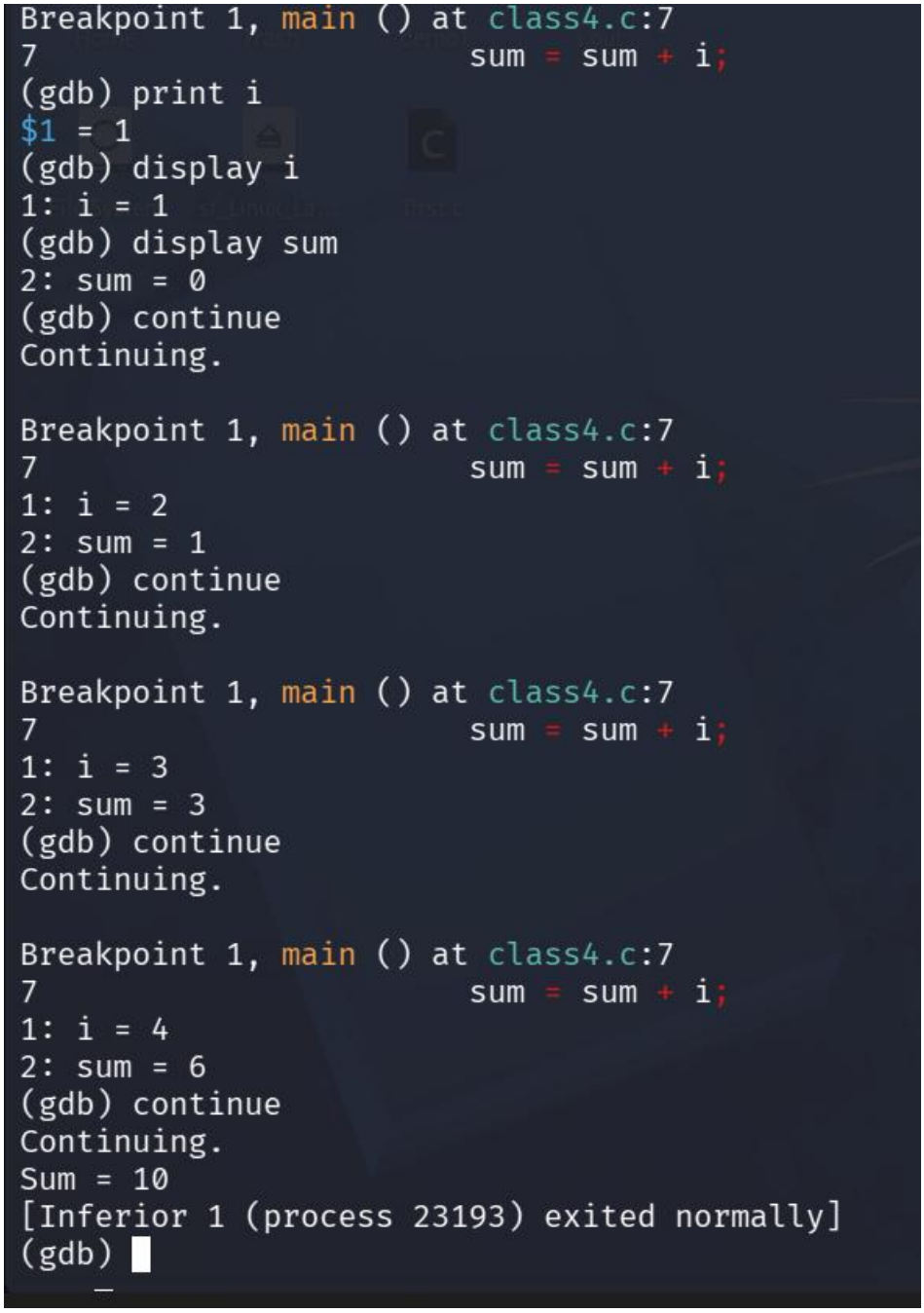
```
(gdb) list
1       #include <stdio.h>
2       int main()
3       {
4               int i, sum=0;
5               for(i=1;i<5;i++)
6               {
7                       sum = sum + i;
8               }
9               printf("Sum = %d\n", sum);
10              return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file class4.c, line 7.
(gdb) run
Starting program: /home/kali/class4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at class4.c:7
7                       sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at class4.c:7
7                       sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
```

EXPERIMENT 3

Study of C Programming Debugging Tools



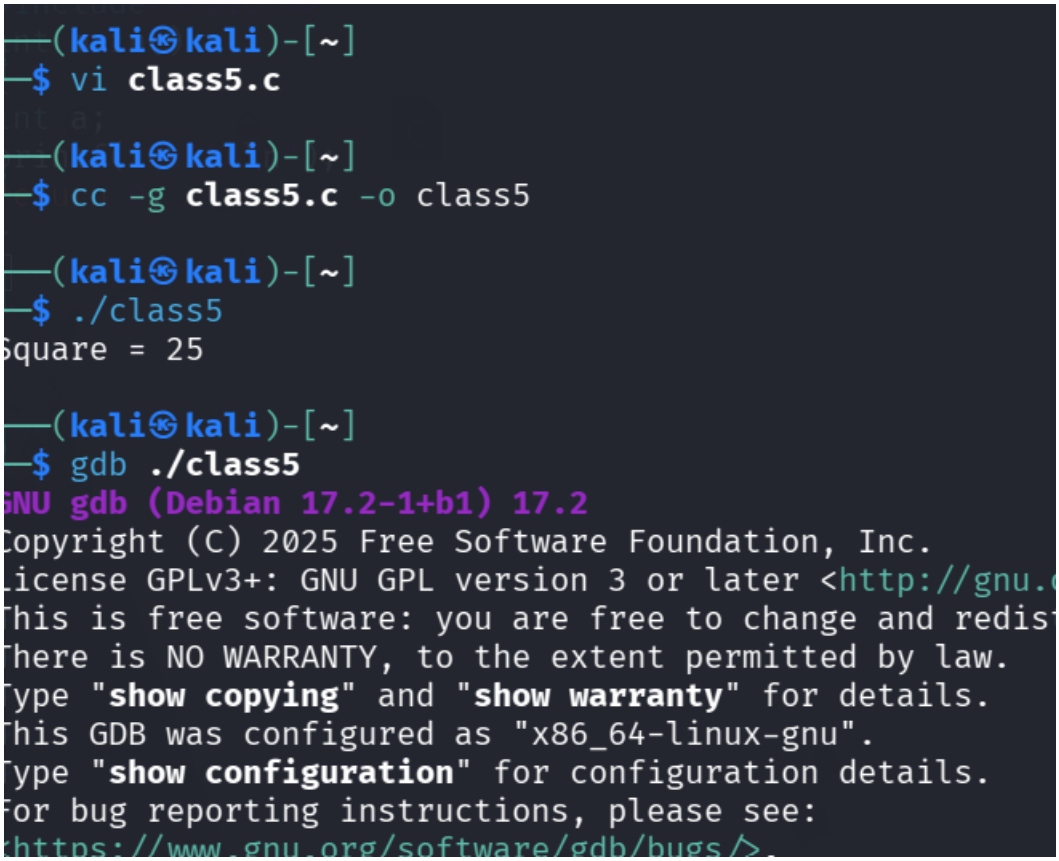
Conclusion: The loop terminates when i reaches 5 without executing the loop body for 5. The loop condition should be changed from i < 5 to i <= 5.

Case 5: Debugging Functions (next vs. step)

• next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.

• step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

OPTION A – Stepping over with next



EXPERIMENT 3

Study of C Programming Debugging Tools

```
┌──(kali㉿kali)-[~]
└─$ gdb ./class5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./class5...
(gdb) break main
Breakpoint 1 at 0x1156: file class5.c, line 10.
(gdb) run
Starting program: /home/kali/class5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at class5.c:10
10              int result, n=5;
(gdb) next
11              result= square(n);
(gdb) next
12              printf("Square = %d\n", result);
(gdb) next
```

```
(gdb) next
12              printf("Square = %d\n", result);
(gdb) next
Square = 25
13              return 0;
(gdb) print result
$1 = 25
```

OPTION B :stepping into with step:

EXPERIMENT 3

Study of C Programming Debugging Tools

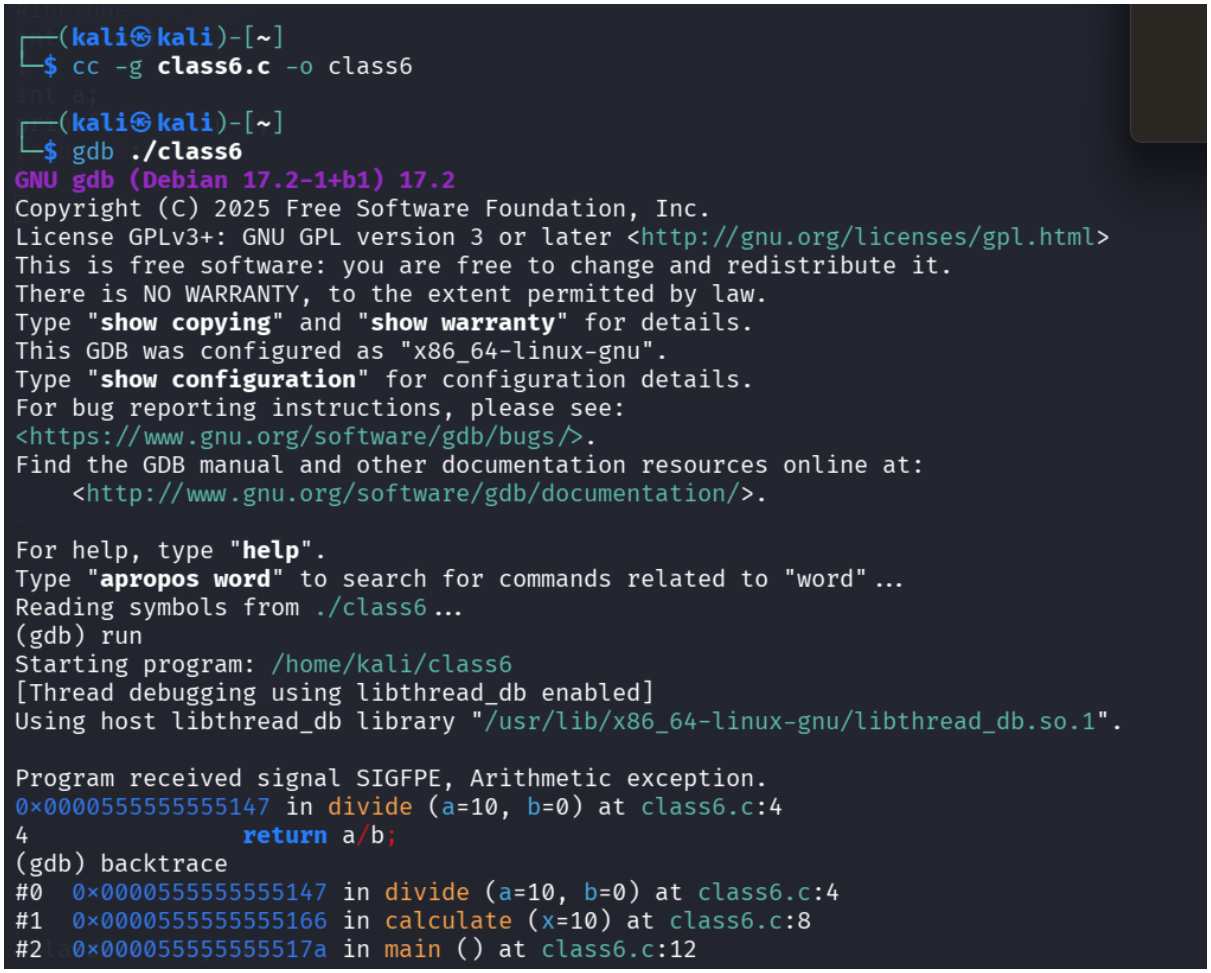
```
Breakpoint 1, main () at class5.c:10
10              int result, n=5;
(gdb) next
11              result= square(n);
(gdb) step
square (x=5) at class5.c:5
5               result1 = x*x;
(gdb) print x
$1 = 5
(gdb) next
6               return result1;
(gdb) print result1
$2 = 25
(gdb)
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

EXPERIMENT 3

Study of C Programming Debugging Tools



Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behaviour or memory corruption.

EXPERIMENT 3

Study of C Programming Debugging Tools

```
┌──(kali㉿kali)-[~]
└─$ gdb ./class7
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./class7...
(gdb) break 8
Breakpoint 1 at 0x114a: file class7.c, line 8.
(gdb) run
Starting program: /home/kali/class7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at class7.c:8
8               scanf("%d", &a[i]);
(gdb) print &a[0]
$1 = (int *) 0x7fffffffdd30
(gdb) print &a[4]
$2 = (int *) 0x7fffffffdd40
(gdb) print &a[5]
$3 = (int *) 0x7fffffffdd44
(gdb)
```

Correct loop conditions ; for(i=0;i<5;i++)

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

EXPERIMENT 3

Study of C Programming Debugging Tools

```
┌──(kali㉿kali)-[~]
└─$ gdb ./class8
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./class8...
(gdb) run
Starting program: /home/kali/class8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at class8.c:5
5               *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print p
$2 = (int *) 0x0
(gdb) print*p
Cannot access memory at address 0x0
(gdb)
```

Q2 GNU project and fitting of GDB into the GNU ecosystem.

GNU Project:

The **GNU Project** is a free and open-source software initiative started by **Richard Stallman in 1983**. Its main objective is to develop a complete Unix-like operating system that gives users the freedom to run, study, modify, and distribute software. The GNU Project has developed many important software tools, compilers, libraries, and utilities that are widely used in Linux systems.

GNU Debugger (GDB):

The **GNU Debugger (GDB)** is one of the software tools developed as part of the GNU Project. It is used to debug C and C++ programs by allowing programmers to execute programs step by step, set breakpoints, inspect variables, and identify logical or runtime errors. GDB helps developers locate and fix bugs efficiently, making it an essential debugging tool in the GNU software ecosystem.

EXPERIMENT 3

Study of C Programming Debugging Tools

Q3 Define Shell and Kernel and list three common Linux shells

Kernel

The **Kernel** is the core component of the operating system. It acts as an interface between the computer hardware and software. The kernel manages system resources such as CPU, memory, storage, input/output devices, and process scheduling. It ensures that different programs can safely share hardware resources.

Shell

The **Shell** is a command-line interpreter that allows users to interact with the operating system. It accepts user commands, interprets them, and communicates with the kernel to execute the requested operations. The shell provides an interface for running programs, managing files, and executing scripts.

Common Linux Shells

1. **Bash (Bourne Again Shell)** – The default shell in most Linux distributions. It is widely used because of its scripting capabilities and ease of use.
2. **Zsh (Z Shell)** – An advanced shell that provides improved auto-completion, themes, plugins, and enhanced customization compared to Bash.
3. **Fish (Friendly Interactive Shell)** – A user-friendly shell with syntax highlighting, command suggestions, and auto-completion, making it easier for beginners.

Q4. Difference between Breakpoint, Watchpoint, Catchpoint, and Tracepoint.

Breakpoint:
A breakpoint is a debugging point set at a specific line of code or function. When the program reaches that location, execution automatically pauses, allowing the programmer to inspect variable values, check the program flow, and identify errors before continuing execution.

Watchpoint:
A watchpoint monitors the value of a variable or memory location during program execution. The debugger stops the program automatically whenever the monitored value changes. It is useful for finding unexpected modifications to variables.

Catchpoint:
A catchpoint is used to stop program execution when a specific event occurs,

such as an exception, signal, or library loading. It helps programmers identify and analyze runtime events that may cause program failures.

Tracepoint:
A tracepoint records information about the program's execution without stopping it. It collects debugging data while the program continues to run, making it useful for analyzing the behavior of large or complex applications where pausing execution is not desirable.

Result

The different types of errors in C programming, namely **syntax errors, compilation/type errors, linker errors, runtime errors, and logical errors**, were successfully studied and demonstrated. Compiler warning options (-Wall and -Wextra), printf() debugging, and the **GNU Debugger (GDB)** were used to identify and analyze program errors. The experiment helped in understanding the use of breakpoints, variable inspection, and other debugging techniques for locating and correcting bugs, thereby improving the accuracy and reliability of C programs.

.....

NAME – AYUSHI KUMAWAT

SAP ID – 590032252

BATCH – 16

COURSE – B.TECH CSE 2026- 2030